

This WasteLesss Electronic Project Archive Contains the Following Items

5/11/2025

- WasteLesss Electronic Project Overview.pdf - this contain the breakdown of all the items contained in the Archive
- Documentation – This contains the entire projects documentation
 - Reports 1-3
 - Presentation_slides Demo1 & Demo2
 - project_management - contains the documentation relative to project management
 - WasteLesss_User_Manual – this is the user manual for the WasteLesss system
- Design – This contains all the technical diagrams
- Code – This contains the entire projects code
 - .vscode – this file contains settings for the alerts
 - config –
 - allowedOrigins.js – Contains logic to handle CORS
 - corsOptions.js – Contains logic to handle CORS
 - roles_list.js – Contains logic for user roles
 - controllers – contains logic that controls and sends backend requests (GET/POST/DELETE/PATCH/PUT) for all tabs – validates all inputs and talks directly with the database to process data.
 - agingInventoryController.js – for the agingInventory endpoint
 - alertController.js – for the alerts
 - authController.js – for the user authentication
 - companyDivisionController.js – for the companyDivision endpoint
 - customerController.js – for the customer endpoint
 - emailController.js – for the manual emails only
 - inventoryController.js – for the inventory endpoint
 - locationController.js – for the location endpoint
 - logoutController.js – for the user logout
 - orderController.js – for the orders endpoint
 - orderItemsController.js – for the orderItems endpoint
 - refreshTokenController.js – for the user authentication
 - registerController.js – for the user authentication
 - rolesController.js – for the user authentication
 - shippingController.js – for the shipping endpoint
 - userAccountController.js – for the user accounts / user authentication
 - vendorController.js – for the vendor endpoint
 - middleware –
 - credentials.js - Contains logic to handle CORS
 - errorHandler.js – Central error handling for catching errors and throws responses

- logEvents.js - Logs request or error events for auditing/debugging
 - verifyJWT.js - Verifies JSON Web Tokens.
 - verifyRoles.js - Checks user roles for authorization.
- model – this holds the database schema
 - sql –
 - schema.sql – Holds all the logic to create the database
 - README4.txt – How to Run Data Collection Scripts
- node_modules – Contains npm packages required to run node.js source code.
- public –
 - css –
 - style.css - Adds visual styling to your HTML elements
 - js – Enhances user interaction, dynamic updates, and handles frontend logic. Listens for user actions (clicks, form submissions), makes API calls, and updates the page dynamically for all tabs
 - agingInventory.js – for the agingInventory table / tab
 - alertEmailJob.js – for automatic alerts that send every day
 - client.js – for user authentication
 - companyDivision.js – for the companyDivision table / tab
 - customer.js – for the customer table / tab
 - global.js – for the user authentication
 - inventory.js – for the inventory table / tab and holds logic for alerts
 - location.js – for the location table / tab
 - login.js – for the user authentication
 - order.js – for the order table / tab
 - orderItems.js - for the orderItems table / tab
 - register.js – for user authentication and roles
 - shipping.js – for the shipping table / tab
 - vendor.js – for the vendor table / tab
 - worker.js – for the user authentication / roles
- routes – Contains logic that controls backend requests (GET/POST/DELETE/PATCH/PUT) for all tabs - Connects incoming HTTP requests (like GET, POST, PUT, DELETE) to specific controller logic that processes them.
 - alertRoutes.js – for the alerts endpoint
 - authRoutes.js – for the user authentication
 - companyDivisionRoutes.js – for the companyDivision endpoint
 - customerRoutes.js – for the customers endpoint
 - emailRoutes.js – for the email endpoint
 - inventoryRoutes.js – for the inventory endpoint and push notifications
 - locationRoutes.js – for the location endpoint

- logoutRoutes.js – for the user logout
 - orderItemsRoutes.js – for the orderItems endpoint
 - orderRoutes.js – for the orders endpoint
 - refreshRoutes.js – for the user authentications
 - registerRoutes.js – for the user authentication
 - root.js – for the – main router for the homepage (index.html)
 - shippingRoutes.js – for the shipping endpoint
 - subscribeRoute.js – for the user authentication and push notifications
 - userAccountRoutes.js – for the user authentication
 - vendorRoutes.js – for the vendor end
- views – Holds frontend templates (static or dynamic) used to render web pages. Defines the layout, text, forms, buttons, and placeholders that may be filled with real data for all tabs
 - 404.html – page not found
 - agingInventory.html – agingInventory tab
 - companyDivision.html – companyDivision tab
 - customer.html – customers tab
 - index.html – the home page
 - inventory.html – inventory and alerts tab
 - location.html – location tab
 - login.html – the login page
 - order.html – order tab
 - orderItems.html – the orderItems tab
 - register.html – register for new account screen
 - shipping.html – the shipping tab
 - unauthorized.html – unauthorized page
 - vendor.html – the vendor tab
- db.js – Contains all the database connection configuration to use PostgreSQL.
- package-lock.json – Records the exact versions of every installed npm package (node-module).
- package.json – Contains project metadata and dependency manager
- server.js - Initializes the Express app, connects to the database, sets up middleware/routes, and starts the server listening on a specified port.
- README1.txt – this contains the walkthrough to use the WasteLesss system.
- Tests
 - datatestingschema.sql – testing schema for testing database
 - test-db.js – test program to ensure backed and database are connected
 - README2.txt – this contains the walkthrough for unit testing of the API endpoints.

- README3.txt – this contains the walkthrough for integration testing. It uses the inventory tab as an example.
- Logs_Unit_Testing
 - reqLog.txt – this shows the output after running the test procedure outlined in the README2.txt.
 - errLog.txt – this shows errors after running the test procedure outlined in the README2.txt.